Sliding Window and Two Pointer Combined Problems

1. Length of Longest Substring without any Repeating Character

You are given a string S.
Your task is to find the **length of the longest substring** that contains **no repeating characters**.
A substring must be **continuous**, and all characters inside it must be **distinct**.

Example Explanation

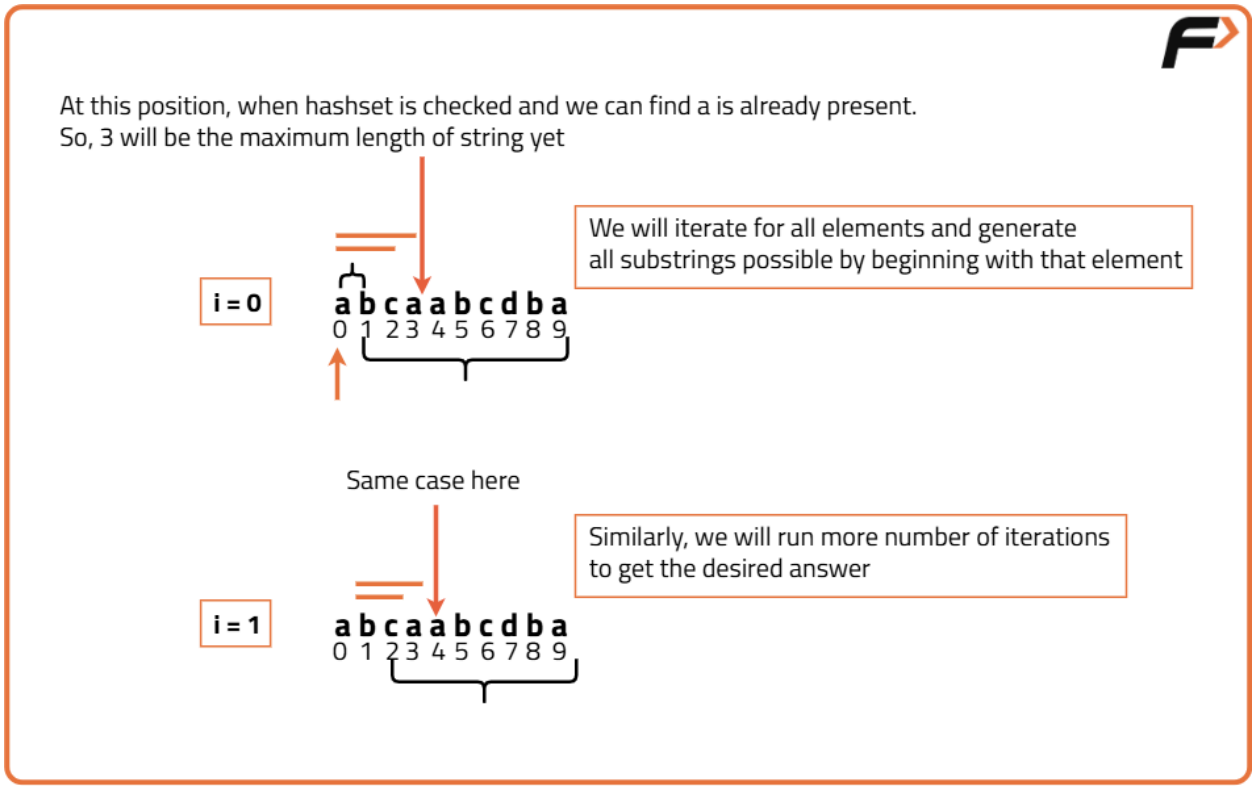
Example 1
Input: S = "abcddabac"
The substring "abcd" has all unique characters and length 4.
No longer substring without repetition exists.
Output: 4

Example 2
Input: S = "aaabbbccc"
Possible longest substrings without repeating characters are "ab" and "bc", both of length 2.
Output: 2

Approach 1 (Brute Force)

Algorithm

- Consider every possible starting index of a substring.
- For each starting index, extend the substring character by character.
- Use a hash array to track whether a character has already appeared.
- If a repeated character is found, stop extending the substring.
- Keep updating the maximum length found.
- Return the maximum length at the end.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestNonRepeatingSubstring(string &s) {
        int n = s.size();
        int maxLen = 0;

        for(int i = 0; i < n; i++) {
            vector<int> hash(256, 0);
            for(int j = i; j < n; j++) {
                if(hash[s[j]] == 1) break;
                hash[s[j]] = 1;
                int len = j - i + 1;
                maxLen = max(maxLen, len);
            }
        }
```

```
        return maxLen;
    }
};

int main() {
    string s = "abcddabac";
    Solution sol;
    cout << sol.longestNonRepeatingSubstring(s);
    return 0;
}
```

Complexity Analysis

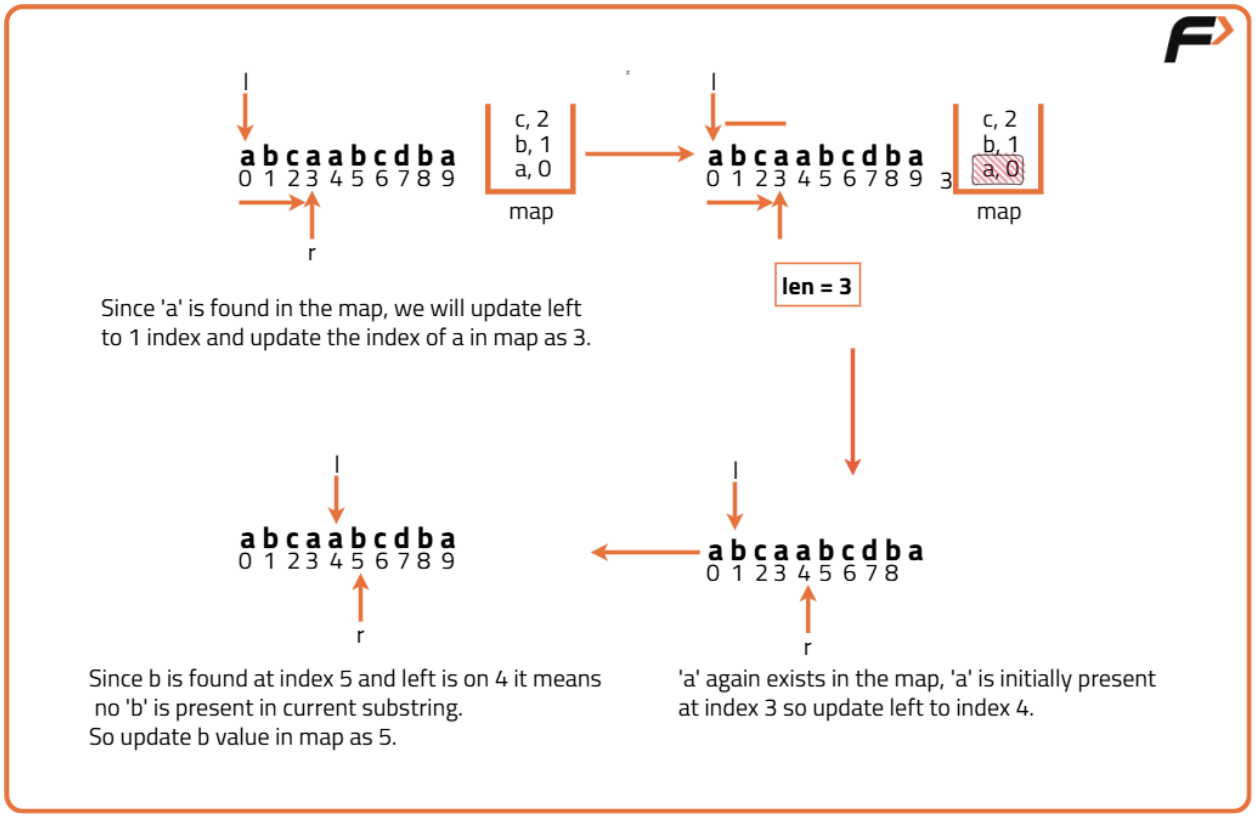
- **Time Complexity:** $O(n^2)$
  Nested loops check all possible substrings.

- **Space Complexity:** $O(1)$
  Fixed-size hash array of 256 characters.

Approach 2 (Optimal – Sliding Window)

Algorithm

- Use two pointers `l` (left) and `r` (right) to maintain a window of unique characters.
- Maintain a hash array storing the **last index** of each character.
- Move the right pointer forward.
- If a character is repeated, move the left pointer just after its last occurrence.
- Update the maximum window length.
- Continue until the end of the string.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestNonRepeatingSubstring(string& s) {
        int n = s.size();
        int hash[256];
        for(int i = 0; i < 256; i++) hash[i] = -1;

        int l = 0, r = 0, maxLen = 0;

        while(r < n) {
            if(hash[s[r]] != -1) {
                l = max(l, hash[s[r]] + 1);
            }
            int len = r - l + 1;
            maxLen = max(maxLen, len);
```

```
            hash[s[r]] = r;
            r++;
        }
        return maxLen;
    }
};

int main() {
    string s = "abcddabac";
    Solution sol;
    cout << sol.longestNonRepeatingSubstring(s);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n)`
  Each character is processed once using two pointers.

- **Space Complexity:** `O(1)`
  Fixed-size hash array of 256 characters.

2. Max Consecutive Ones III

You are given a binary array `nums` (containing only `0` and `1`) and an integer `k`.
You are allowed to flip **at most k zeros into ones**.
Your task is to return the **maximum number of consecutive 1s** that can be obtained after performing at most k flips.

Example Explanation

Example 1

Input:
`nums = [1,1,1,0,0,0,1,1,1,1,0], k = 3`

By flipping the zeros at indices `3, 4, 5`, the array becomes:
`[1,1,1,1,1,1,1,1,1,1,0]`

The longest consecutive sequence of 1s is `10`.
Output: `10`

Example 2

Input:
`nums = [0,0,1,1,1,0,1,1,1,0,0,0,0,1,1,1,1], k = 3`

After flipping suitable zeros, the longest consecutive sequence of 1s is `9`.
Output: `9`

Approach 1 (Brute Force)

Algorithm

- Try every possible subarray.
- For each subarray, count how many zeros it contains.
- If the number of zeros is less than or equal to `k`, the subarray is valid.
- Track the maximum length among all valid subarrays.
- Return the maximum length found.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int longestOnes(vector<int>& nums, int k) {
        int n = nums.size();
        int maxLen = 0;

        for(int i = 0; i < n; i++) {
            int zeros = 0;
            for(int j = i; j < n; j++) {
                if(nums[j] == 0) zeros++;
                if(zeros > k) break;
                maxLen = max(maxLen, j - i + 1);
            }
        }
        return maxLen;
    }
};

int main() {
    vector<int> nums = {1,1,1,0,0,0,1,1,1,1,0};
    int k = 3;
    Solution sol;
    cout << sol.longestOnes(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n^2)$ due to nested loops.
- **Space Complexity:** $O(1)$ since no extra space is used.

Approach 2 (Better – Sliding Window)

Algorithm

- Use a sliding window with two pointers `left` and `right`.
- Expand the window using `right`.
- Count zeros inside the window.
- If zeros exceed k, shrink the window from the left.
- Update the maximum window size after each step.
- Return the maximum length found.

Right pointer ek step me sirf 1 zero add kar sakta hai.

So:

- `zerocount` **maximum 1 se hi badh sakta hai**
- Isliye window **sirf 1 step invalid hoti hai**
- Usko theek karne ke liye **1 step shrink bhi kaafi hota hai**

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestOnes(vector<int>& nums, int k) {
        int left = 0, zeros = 0, maxLen = 0;

        for(int right = 0; right < nums.size(); right++) {
            if(nums[right] == 0) zeros++;

            while(zeros > k) {
```

```
                if(nums[left] == 0) zeros--;
                left++;
            }
            maxLen = max(maxLen, right - left + 1);
        }
        return maxLen;
    }
};

int main() {
    vector<int> nums = {1,1,1,0,0,0,1,1,1,1,0};
    int k = 3;
    Solution sol;
    cout << sol.longestOnes(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(1)`

Approach 3 (Optimal – Optimized Sliding Window)

Algorithm

- Use two pointers `left` and `right`.
- Maintain a count of zeros in the current window.
- Move `right` forward each step.
- If zero count exceeds k, move `left` forward once and adjust zero count.

- Update maximum window length at each step.
- This avoids an inner loop and keeps logic simple.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestOnes(vector<int>& nums, int k) {
        int left = 0, zerocount = 0, maxLen = 0;

        for(int right = 0; right < nums.size(); right++) {
            if(nums[right] == 0) zerocount++;

            if(zerocount > k) {
                if(nums[left] == 0) zerocount--;
                left++;
            }
            maxLen = max(maxLen, right - left + 1);
        }
        return maxLen;
    }
};

int main() {
    vector<int> nums = {1,1,1,0,0,0,1,1,1,1,0};
    int k = 3;
    Solution sol;
    cout << sol.longestOnes(nums, k);
    return 0;
}
```

- **Time Complexity:** `O(n)`

- **Space Complexity:** `O(1)`

Doubt clear karne k liye example: nums = [1,1,1,0,0,0,1]
k = 2

window thodi der invalid reh sakti hai,
lekin wo kabhi answer ko bigaadti nahi.

3. Fruit Into Baskets

You are given an array `fruits` where `fruits[i]` represents the type of fruit on the `i-th` tree.
You have **only two baskets**, and **each basket can store only one type of fruit**, but in unlimited quantity.

You can start picking fruits from **any tree**, but once you start, you must move **only to the right**, picking **exactly one fruit from each tree**.
If you encounter a fruit type that cannot fit into your two baskets, you must stop.

Your task is to return the **maximum number of fruits** you can collect following these rules.

Example Explanation

Example 1
Input: `fruits = [1, 2, 1]`

We start from index `0`:

- Basket 1 → fruit 1
- Basket 2 → fruit 2
- Next fruit is 1, which already exists in basket

Total fruits collected = 3

Output: 3

Example 2
Input: `fruits = [1, 2, 3, 2, 2]`

Best starting point is index 1:

- Fruits collected: `[2, 3, 2, 2]`
- Only two types (2 and 3)

Total fruits collected = 4

Output: 4

Approach 1 (Brute Force)

Algorithm

- Try starting from every index.
- From each start index, move right and collect fruits.
- Keep track of fruit types using a map or set.
- Stop when more than 2 fruit types are encountered.
- Track the maximum number of fruits collected.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    int totalFruit(vector<int>& fruits) {
        int n = fruits.size();
        int maxFruits = 0;

        for(int start = 0; start < n; start++) {
            unordered_map<int,int> basket;
            int count = 0;

            for(int end = start; end < n; end++) {
                basket[fruits[end]]++;
                if(basket.size() > 2) break;
                count++;
            }
            maxFruits = max(maxFruits, count);
        }
        return maxFruits;
    }
};

int main() {
    vector<int> fruits = {1,2,1};
    Solution sol;
    cout << sol.totalFruit(fruits);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$ (at most 3 fruit types in map)

Approach 2 (Better – Sliding Window with Hash Map)

Algorithm

- Use a sliding window with two pointers `left` and `right`.
- Maintain a hash map to count fruit types in the current window.
- Expand window to the right.
- If fruit types exceed 2, shrink window from the left.
- Track the maximum window size.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int totalFruit(vector<int>& fruits) {
        unordered_map<int,int> basket;
        int left = 0, maxFruits = 0;

        for(int right = 0; right < fruits.size(); right++) {
            basket[fruits[right]]++;

            while(basket.size() > 2) {
                basket[fruits[left]]--;
                if(basket[fruits[left]] == 0)
                    basket.erase(fruits[left]);
                left++;
            }
            maxFruits = max(maxFruits, right - left + 1);
        }
        return maxFruits;
    }
};

int main() {
    vector<int> fruits = {1,2,3,2,2};
```

```
    Solution sol;
    cout << sol.totalFruit(fruits);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$ (map holds at most 2 keys)

Approach 3 (Optimal – Without Hash Map)

Algorithm

- Track only the **last two fruit types**.
- Maintain:
    - `lastFruit`
    - `secondLastFruit`
    - `lastFruitCount`
    - `currentWindowSize`
- If current fruit matches one of the two, extend window.
- Otherwise, shrink window to last fruit streak + current fruit.
- Update maximum length.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int totalFruit(vector<int>& fruits) {
        int lastFruit = -1, secondLastFruit = -1;
        int lastCount = 0, currLen = 0, maxLen = 0;

        for(int fruit : fruits) {
            if(fruit == lastFruit || fruit == secondLastFruit)
                currLen++;
            else
                currLen = lastCount + 1;

            if(fruit == lastFruit)
                lastCount++;
            else {
                lastCount = 1;
                secondLastFruit = lastFruit;
                lastFruit = fruit;
            }

            maxLen = max(maxLen, currLen);
        }
        return maxLen;
    }
};

int main() {
    vector<int> fruits = {1,2,1,2,3};
    Solution sol;
    cout << sol.totalFruit(fruits);
    return 0;
}
```

- **Time Complexity:** O(n)
- **Space Complexity:** O(1)

4. Longest Repeating Character Replacement

You are given a string s consisting of **uppercase English letters** and an integer k.
You are allowed to **replace at most k characters** in the string with any other uppercase letter.

After performing at most k replacements, you need to find the **length of the longest substring that contains only one repeating character**.

Example Explanation

Example 1
Input: `s = "BAABAABBBAAA", k = 2`

We can replace:

- B at index `0` → `A`
- B at index 3 → `A`

New string becomes:
`AAAAAABBBAAA`

The longest substring with the same character is `"AAAAAA"`
Length = 6

Output: 6

Example 2
Input: `s = "AABABBA", k = 1`

We can replace one character to form `"AABBBBA"`
The longest repeating substring is `"BBBB"`

Output: 4

Brute Force Approach

Algorithm

- Try **all possible substrings**.
- For each substring:
    - Count frequency of characters.
    - Find the most frequent character.
    - Calculate replacements needed = `substring length - max frequency`.
- If replacements ≤ k, update the answer.
- Return the maximum valid length found.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int characterReplacement(string s, int k) {
        int n = s.size();
        int maxLength = 0;

        for(int i = 0; i < n; i++) {
            vector<int> freq(26, 0);
            int maxFreq = 0;

            for(int j = i; j < n; j++) {
                freq[s[j] - 'A']++;
                maxFreq = max(maxFreq, freq[s[j] - 'A']);
                int windowLen = j - i + 1;
```

```
                if(windowLen - maxFreq <= k) {
                    maxLength = max(maxLength, windowLen);
                }
            }
        }
        return maxLength;
    }
};

int main() {
    Solution sol;
    string s = "AABABBA";
    int k = 1;
    cout << sol.characterReplacement(s, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$ (fixed 26-size frequency array)

Better Approach (Sliding Window)

Algorithm

- Use two pointers `left` and `right` to form a window.
- Maintain frequency of characters inside the window.
- Track the **maximum frequency** character in the window.
- If `(window size - max frequency) > k`, shrink window from left.
- Keep updating the maximum window size.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int characterReplacement(string s, int k) {
        unordered_map<char,int> freq;
        int left = 0, maxFreq = 0, maxLen = 0;

        for(int right = 0; right < s.size(); right++) {
            freq[s[right]]++;
            maxFreq = max(maxFreq, freq[s[right]]);

            while((right - left + 1) - maxFreq > k) {
                freq[s[left]]--;
                left++;
            }

            maxLen = max(maxLen, right - left + 1);
        }
        return maxLen;
    }
};

int main() {
    Solution sol;
    string s = "AABABBA";
    int k = 1;
    cout << sol.characterReplacement(s, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n)$

- **Space Complexity:** $O(1)$ (at most 26 characters)

Optimal Approach (Optimized Sliding Window)

Algorithm

- Use a **fixed-size frequency array (26)** instead of a map.
- Maintain `maxCount` = highest frequency in the window.
- Expand the window to the right.
- If replacements needed exceed `k`, move left pointer.
- We **do not reduce `maxCount`** while shrinking, because it does not affect correctness.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int characterReplacement(string s, int k) {
        vector<int> freq(26, 0);
        int left = 0, maxCount = 0, maxLen = 0;

        for(int right = 0; right < s.size(); right++) {
            freq[s[right] - 'A']++;
            maxCount = max(maxCount, freq[s[right] - 'A']);

            if((right - left + 1) - maxCount > k) {
                freq[s[left] - 'A']--;
                left++;
            }
            maxLen = max(maxLen, right - left + 1);
        }
```

```
        return maxLen;
    }
};

int main() {
    Solution sol;
    string s = "AABABBA";
    int k = 1;
    cout << sol.characterReplacement(s, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(1)`

5. Binary Subarray With Sum

You are given a **binary array** `nums` (containing only `0` and `1`) and an integer `goal`.
You need to count how many **non-empty contiguous subarrays** have a sum **exactly equal to `goal`**.

A subarray means a continuous part of the array.

Example Explanation

Example 1
Input: `nums = [1,0,0,1,1,0]`, `goal = 2`

Valid subarrays with sum = 2 are:

- `[1,0,0,1]`
- `[0,0,1,1]`
- `[0,1,1]`
- `[1,1]`
- `[1,1,0]`
- `[0,0,1,1,0]`

Total = **6**

Output: 6

Example 2
Input: `nums = [0,0,0,0,0,0]`, `goal = 0`

Every possible subarray has sum `0`.
Number of subarrays = `n(n+1)/2 = 6*7/2 = 21`

Output: 21

Brute Force Approach

Algorithm

- Try **all possible subarrays** using two loops.
- Fix a starting index.
- Expand the ending index and keep adding elements to a running sum.
- If the sum equals `goal`, increase the count.

- After checking all subarrays, return the count.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numSubarraysWithSum(vector<int>& nums, int goal) {
        int count = 0;
        int n = nums.size();

        for(int i = 0; i < n; i++) {
            int sum = 0;
            for(int j = i; j < n; j++) {
                sum += nums[j];
                if(sum == goal) {
                    count++;
                }
            }
        }
        return count;
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1,0,1,0,1};
    int goal = 2;
    cout << sol.numSubarraysWithSum(nums, goal);
    return 0;
}
```

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$

Better Approach (Prefix Sum + Hash Map)

Algorithm

- Use a prefix sum technique.
- Maintain a hashmap that stores how many times a prefix sum has appeared.
- At any index, if (`currentSum - goal`) exists in the map, it means valid subarrays end here.
- Add its frequency to the answer.
- Update the map with the current prefix sum.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numSubarraysWithSum(vector<int>& nums, int goal) {
        unordered_map<int,int> mp;
        mp[0] = 1;

        int sum = 0, count = 0;

        for(int x : nums) {
            sum += x;
            if(mp.find(sum - goal) != mp.end()) {
                count += mp[sum - goal];
            }
            mp[sum]++;
        }
        return count;
    }
};
```

```
int main() {
    Solution sol;
    vector<int> nums = {1,0,1,0,1};
    int goal = 2;
    cout << sol.numSubarraysWithSum(nums, goal);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(n)`

Optimal Approach (Sliding Window using AtMost Trick)

This approach works because the array contains **only non-negative numbers (0 and 1)**.

- Count subarrays with sum **at most `goal`**
- Count subarrays with sum **at most `goal - 1`**
- Subtract them to get subarrays with sum **exactly `goal`**

Why it works:

- Subarrays with exact sum = `goal`
- = subarrays with sum ≤ `goal` minus subarrays with sum ≤ `goal-1`

Steps:

- Use a sliding window with two pointers.
- Expand right pointer and add to sum.
- If sum exceeds limit, shrink from left.
- Add window size to count.
- Final answer = `atMost(goal) - atMost(goal-1)`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numSubarraysWithSum(vector<int>& nums, int goal) {
        return atMost(nums, goal) - atMost(nums, goal - 1);
    }

private:
    int atMost(vector<int>& nums, int k) {
        if(k < 0) return 0;

        int left = 0, sum = 0, count = 0;

        for(int right = 0; right < nums.size(); right++) {
            sum += nums[right];

            while(sum > k) {
                sum -= nums[left];
                left++;
            }
            count += (right - left + 1);
        }
        return count;
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1,0,1,0,1};
    int goal = 2;
    cout << sol.numSubarraysWithSum(nums, goal);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(1)`

WHY `(right - left + 1)` ?

- Window `[left … right]` valid hai (sum ≤ k)

Toh:

- `[right … right]`
- `[right-1 … right]`
- `[right-2 … right]`
- ...
- `[left … right]`

Total = right - left + 1 subarrays

Aur **sabka sum ≤ k** hoga

6. Count Number of Nice Subarrays

You are given an integer array `nums` and an integer `k`.
A subarray is called **nice** if it contains **exactly k odd numbers**.
Your task is to count the number of such nice subarrays.

A subarray is a **continuous** part of the array.

Example Explanation

Example 1
Input: `nums = [1, 1, 2, 1, 1]`, `k = 3`

Odd numbers appear at indices: `0, 1, 3, 4`

Nice subarrays with exactly 3 odd numbers:

- `[1, 1, 2, 1]`
- `[1, 2, 1, 1]`

Output: 2

Example 2
Input: `nums = [4, 8, 2]`, `k = 1`

There are no odd numbers in the array, so no subarray can contain 1 odd number.

Output: `0`

Brute Force Approach

Algorithm

- Try every possible subarray.
- For each starting index, extend the subarray to the right.
- Count how many odd numbers are in the current subarray.
- If odd count becomes greater than `k`, stop expanding.

- If odd count is exactly k, count it as a nice subarray.
- Return the total count.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int numberOfSubarrays(vector<int>& nums, int k) {

        int count = 0;


        for(int start = 0; start < nums.size(); start++) {

            int oddCount = 0;


            for(int end = start; end < nums.size(); end++) {

                if(nums[end] % 2 != 0)

                    oddCount++;


                if(oddCount > k)

                    break;


                if(oddCount == k)
```

```
                    count++;
                }
            }
            return count;
        }
};


int main() {
    Solution sol;
    vector<int> nums = {1,1,2,1,1};
    int k = 3;
    cout << sol.numberOfSubarrays(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
- **Space Complexity:** $O(1)$

Better Approach (Prefix Odd Count + Hash Map)

- Convert the problem into counting prefixes.

- Maintain a running count of odd numbers seen so far.
- Use a hashmap where:
    - key = number of odd numbers seen
    - value = how many times this count has occurred
- For each index:
    - If current odd count is `oddCount`
    - Any previous prefix with `oddCount - k` forms a nice subarray
- Add its frequency to the answer.
- Update the map.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int numberOfSubarrays(vector<int>& nums, int k) {

        unordered_map<int,int> freq;

        freq[0] = 1;


        int oddCount = 0;

        int result = 0;
```

```
        for(int num : nums) {
            if(num % 2 != 0)
                oddCount++;

            if(freq.count(oddCount - k))
                result += freq[oddCount - k];

            freq[oddCount]++;
        }
        return result;
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1,1,2,1,1};
    int k = 3;
    cout << sol.numberOfSubarrays(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
- **Space Complexity:** `O(N)`

Optimal Approach (Sliding Window – At Most Trick)

Algorithm

- We cannot directly count subarrays with exactly k odd numbers using one sliding window.
- Instead:
    - Count subarrays with **at most k odd numbers**
    - Count subarrays with **at most k−1 odd numbers**
- The difference gives subarrays with **exactly k odd numbers**.

Formula:

```
exactlyK = atMost(k) - atMost(k-1)
```

Helper function `atMost(k)`:

- Use sliding window.
- Expand right pointer.
- Reduce k when encountering odd numbers.
- Shrink window when `k < 0`.
- Add `(right - left + 1)` to result.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int countAtMost(vector<int>& nums, int k) {

        int left = 0, res = 0;


        for(int right = 0; right < nums.size(); right++) {

            if(nums[right] % 2 != 0)

                K--; // K is allowance


            while(k < 0) {

                if(nums[left] % 2 != 0)

                    k++;

                left++;

            }

            res += (right - left + 1);

        }

        return res;

    }
```

```
    int numberOfSubarrays(vector<int>& nums, int k) {
        return countAtMost(nums, k) - countAtMost(nums, k - 1);
    }
};


int main() {
    Solution sol;
    vector<int> nums = {1,1,2,1,1};
    int k = 3;
    cout << sol.numberOfSubarrays(nums, k);
    return 0;
}
```

- **Time Complexity:** `O(N)`
- **Space Complexity:** `O(1)`

7. Number of Substrings Containing All Three Characters

You are given a string `s` that contains only the characters `'a'`, `'b'`, and `'c'`.
Your task is to count how many **substrings** of `s` contain **at least one occurrence of all three characters** `'a'`, `'b'`, and `'c'`.

A substring is a **continuous** part of the string.

Example Explanation

Example 1

Input: `s = "abcba"`

Valid substrings that contain `'a'`, `'b'`, and `'c'`:

- `"abc"`
- `"abcb"`
- `"abcba"`
- `"bcba"`
- `"cba"`

Output: 5

Example 2

Input: `s = "ccabcc"`

Valid substrings:

- `"ccab"`
- `"ccabc"`
- `"ccabcc"`
- `"cab"`
- `"cabc"`
- `"cabcc"`
- `"abc"`

- `"abcc"`

Output: 8

Brute Force Approach

Algorithm

- Try all possible substrings.
- Fix a starting index `i`.
- From `i`, expand the substring to the right using index `j`.
- Maintain a frequency array for characters `'a'`, `'b'`, `'c'`.
- If at any point all three characters are present, count that substring.
- Continue until all substrings are checked.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int numberOfSubstrings(string s) {

        int n = s.size();

        int count = 0;
```

```
        for(int i = 0; i < n; i++) {
            vector<int> freq(3, 0);

            for(int j = i; j < n; j++) {
                freq[s[j] - 'a']++;

                if(freq[0] > 0 && freq[1] > 0 && freq[2] > 0)
                    count++;
            }
        }
        return count;
    }
};

int main() {
    Solution sol;
    string s = "abcba";
    cout << sol.numberOfSubstrings(s);
    return 0;
}
```

- **Time Complexity:** $O(N^2)$
  All possible substrings are checked using two nested loops.

- **Space Complexity:** `O(1)`
  Only a fixed-size frequency array of size 3 is used.

Optimal Approach (Sliding Window)

Algorithm

- Use a sliding window `[left, right]`.
- Maintain a frequency array for `'a'`, `'b'`, `'c'`.
- Expand the window using `right`.
- Once the window contains all three characters:
  - All substrings that start at `left` or before and end at `right` are valid.
  - Add `(n - right)` to the result.
- Shrink the window from the left and continue.
- This avoids checking every substring explicitly.

Jaise hi current window valid ho jaati hai (a, b, c present),

to us window se start hone wale aur right side tak extend hone wale

saare substrings automatically valid hote hain.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {
```

```
public:
    int numberOfSubstrings(string s) {
        int n = s.size();
        vector<int> freq(3, 0);
        int left = 0, result = 0;

        for(int right = 0; right < n; right++) {
            freq[s[right] - 'a']++;

            while(freq[0] > 0 && freq[1] > 0 && freq[2] > 0) {
                result += (n - right);  //left - right to left - n
                freq[s[left] - 'a']--;
                left++;
            }
        }
        return result;
    }
};

int main() {
    Solution sol;
    string s = "abcba";
    cout << sol.numberOfSubstrings(s);
```

```
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each character is processed at most twice (once by `right`, once by `left`).

- **Space Complexity:** `O(1)`
  Constant extra space for the frequency array.

8. Maximum Points You Can Obtain from Cards

You are given an array `cardScore` representing scores of cards placed in a row.
You must choose **exactly k cards**.
At every step, you are allowed to pick **one card either from the beginning or from the end** of the row.

Your task is to return the **maximum total score** you can obtain by picking exactly k cards following these rules.

Example Explanation

Example 1
Input: `cardScore = [1, 2, 3, 4, 5, 6]`, `k = 3`

If we choose the last three cards: `4 + 5 + 6 = 15`, which is the maximum possible score.
Output: 15

Example 2
Input: `cardScore = [5, 4, 1, 8, 7, 1, 3]`, `k = 3`

Best choice:

- Take 5 from start
- Take 4 from start
- Take 3 from end

Total score = `5 + 4 + 3 = 12`
Output: 12

Brute Force Approach

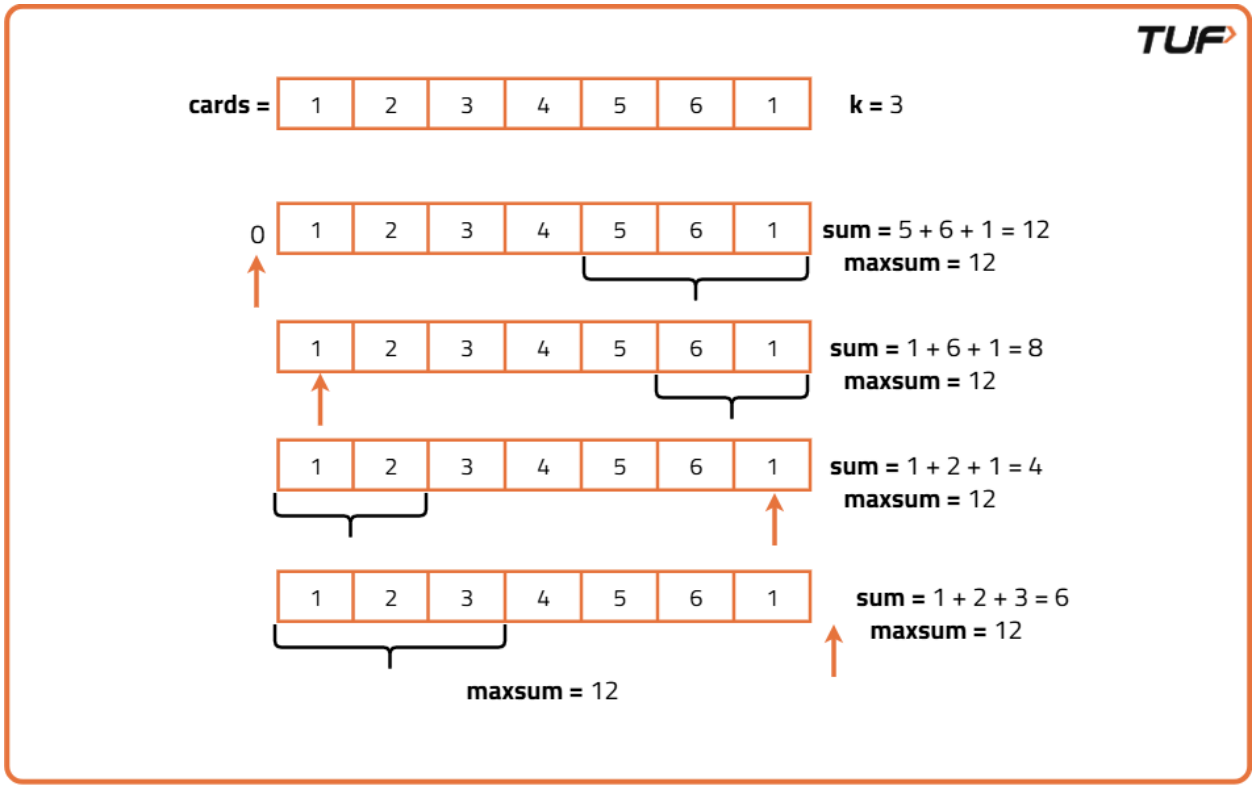
Algorithm

We must pick exactly `k` cards, but they can be taken from either end.
So we try **all possible combinations**:

- Take `i` cards from the start
- Take `k - i` cards from the end
  for all `i` from `0` to `k`.

Steps:

1. Loop `i` from `0` to `k`
2. Sum first `i` cards from the beginning
3. Sum last `k - i` cards from the end
4. Track the maximum sum obtained



Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int maxScore(vector<int>& cardPoints, int k) {

        int n = cardPoints.size();

        int maxSum = 0;


        for(int i = 0; i <= k; i++) {

            int currSum = 0;
```

```
            // Take i cards from the front
            for(int j = 0; j < i; j++)
                currSum += cardPoints[j];

            // Take (k - i) cards from the back
            for(int j = 0; j < k - i; j++)
                currSum += cardPoints[n - 1 - j];

            maxSum = max(maxSum, currSum);
        }
        return maxSum;
    }
};

int main() {
    Solution sol;
    vector<int> cards = {1, 2, 3, 4, 5, 6};
    int k = 3;
    cout << sol.maxScore(cards, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(k^2)$
  For each of k combinations, we sum up to k elements.

- **Space Complexity:** $O(1)$
  Only constant extra variables are used.

Optimal Approach

Algorithm

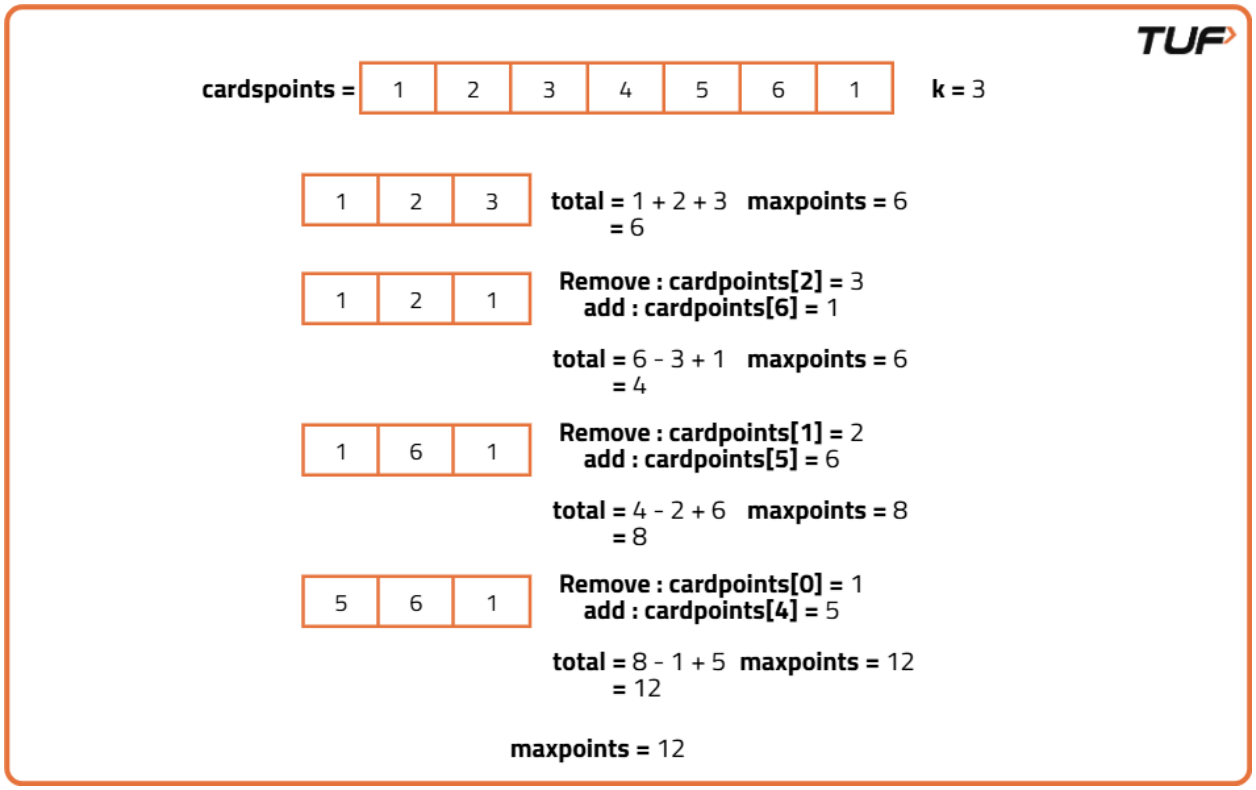
Instead of recalculating sums every time, we use a **sliding window idea**.

Key observation:
If you pick k cards from the ends, it is equivalent to **removing a contiguous subarray of length n − k from the middle** and keeping the rest.

But since the given approach uses front/back shifting, we follow that logic:

Steps:

1. Take all k cards from the **front** and compute the initial sum.
2. Then, step by step:
    - Remove one card from the front
    - Add one card from the back
3. Update the maximum score at each step.
4. Do this k times.



Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int maxScore(vector<int>& cardPoints, int k) {

        int n = cardPoints.size();

        int currSum = 0;


        // Sum of first k cards

        for(int i = 0; i < k; i++)
```

```
            currSum += cardPoints[i];

        int maxSum = currSum;

        // Shift window: remove from front, add from back
        for(int i = 0; i < k; i++) {
            currSum -= cardPoints[k - 1 - i];
            currSum += cardPoints[n - 1 - i];
            maxSum = max(maxSum, currSum);
        }
        return maxSum;
    }
};

int main() {
    Solution sol;
    vector<int> cards = {5, 4, 1, 8, 7, 1, 3};
    int k = 3;
    cout << sol.maxScore(cards, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(k)$
  Initial sum takes k steps, and sliding takes k steps.

- **Space Complexity:** $O(1)$
  Only constant extra variables are used.

9. Longest Substring with At Most K Distinct Characters

You are given a string s and an integer k.
Your task is to find the **length of the longest substring** that contains **at most k distinct characters**.
A substring must be **continuous**.

Example Explanation

Example 1
Input: `s = "aababbcaacc", k = 2`

The substring `"aababb"` contains only two distinct characters (a and b) and has length 6, which is the maximum possible.
Output: 6

Example 2
Input: `s = "abcddefg", k = 3`

The substring `"bcdd"` contains at most three distinct characters (b, c, d) and has length 4.
Output: 4

Brute Force Approach

Algorithm

In this approach, we try **all possible substrings** and check whether each substring contains **at most k distinct characters**.

Steps:

1. Fix a starting index `i`.
2. From `i`, expand the substring to the right using index `j`.
3. Use a frequency map to track distinct characters in the current substring.
4. If the number of distinct characters exceeds k, stop expanding.
5. If valid, update the maximum length.
6. Repeat for all starting indices.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int lengthOfLongestSubstringKDistinct(string s, int k) {

        int maxLength = 0;


        for (int i = 0; i < s.size(); i++) {
```

```
            unordered_map<char, int> freq;

            for (int j = i; j < s.size(); j++) {
                freq[s[j]]++;

                if (freq.size() > k) break;

                maxLength = max(maxLength, j - i + 1);
            }
        }
        return maxLength;
    }
};

int main() {
    Solution sol;
    string s = "aababbcaacc";
    int k = 2;
    cout << sol.lengthOfLongestSubstringKDistinct(s, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n^2)$
  We try all possible substrings using two nested loops.

- **Space Complexity:** $O(k)$
  The frequency map stores at most k distinct characters.

Optimal Approach

Algorithm

Instead of checking all substrings, we use a **sliding window**.

Idea:

- Maintain a window `[left, right]` with **at most k distinct characters**.
- Expand the window by moving `right`.
- If distinct characters exceed k, shrink the window by moving `left`.
- Track the maximum valid window length.

Steps:

1. Initialize two pointers `left = 0`, `right = 0`.
2. Use a frequency map to store character counts.
3. Expand the window by moving `right`.
4. If distinct characters exceed k, shrink from the left.
5. Update the maximum length for each valid window.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int lengthOfLongestSubstringKDistinct(string s, int k) {

        if (k == 0 || s.empty()) return 0;


        unordered_map<char, int> freq;

        int left = 0, maxLen = 0;


        for (int right = 0; right < s.size(); right++) {

            freq[s[right]]++;


            while (freq.size() > k) {

                freq[s[left]]--;

                if (freq[s[left]] == 0)

                    freq.erase(s[left]);

                left++;

            }


            maxLen = max(maxLen, right - left + 1);
```

```
        }

        return maxLen;

    }

};


int main() {

    Solution sol;

    string s = "aababbcaacc";

    int k = 2;

    cout << sol.lengthOfLongestSubstringKDistinct(s, k);

    return 0;

}
```

Complexity Analysis

- **Time Complexity:** $O(n)$
  Each character is added and removed from the window at most once.

- **Space Complexity:** $O(k)$
  The frequency map stores at most k distinct characters.

10. Subarray with K Different Integers

You are given an integer array `nums` and an integer `k`.
Your task is to count how many **contiguous subarrays** contain **exactly k distinct integers**.
Such subarrays are called **good subarrays**.

A subarray must be continuous, meaning elements are taken from consecutive positions in the array.

Example Explanation

For `nums = [1, 2, 1, 2, 3]` and `k = 2`

The good subarrays with exactly 2 distinct integers are:

- `[1, 2]`
- `[2, 1]`
- `[1, 2]`
- `[2, 3]`
- `[1, 2, 1]`
- `[2, 1, 2]`
- `[1, 2, 1, 2]`

So the answer is 7.

Brute Force Approach

In this approach, we try **all possible subarrays** and count how many distinct integers each subarray contains.

Steps:

1. Loop over all possible starting indices.
2. For each starting index, expand the subarray to the right.
3. Use a frequency map to track how many distinct numbers are present.
4. If the number of distinct integers becomes exactly k, increment the count.
5. If it exceeds k, stop expanding further for that start index.
6. Return the total count.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int subarraysWithKDistinct(vector<int>& nums, int k) {

        int n = nums.size();

        int count = 0;


        for (int i = 0; i < n; i++) {

            unordered_map<int,int> freq;


            for (int j = i; j < n; j++) {
```

```
                freq[nums[j]]++;

                if (freq.size() == k)
                    count++;

                if (freq.size() > k)
                    break;
            }
        }
        return count;
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1,2,1,2,3};
    int k = 2;
    cout << sol.subarraysWithKDistinct(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2 * K)$
  We examine all subarrays and manage a frequency map that can grow up to k elements.

- **Space Complexity:** $O(K)$
  The frequency map stores at most k distinct integers.

Optimal Approach

Algorithm

We use a **sliding window** idea with a mathematical observation:

Number of subarrays with **exactly k distinct integers**
= subarrays with **at most k distinct**
− subarrays with **at most k−1 distinct**

So we:

1. Write a helper function `atMostK()` that counts subarrays with at most K distinct elements.
2. Use sliding window with two pointers.
3. Maintain a frequency map for the window.
4. Expand the window using the right pointer.
5. Shrink the window from the left if distinct elements exceed K.
6. For each valid window, add `(right - left + 1)` to the result.
7. Final answer = `atMostK(k) - atMostK(k - 1)`.

Code

```
#include <bits/stdc++.h>
```

```
using namespace std;


class Solution {
public:
    int atMostK(vector<int>& nums, int k) {
        unordered_map<int,int> freq;
        int left = 0, count = 0;


        for (int right = 0; right < nums.size(); right++) {
            if (freq[nums[right]] == 0)
                k--;
            freq[nums[right]]++;


            while (k < 0) {
                freq[nums[left]]--;
                if (freq[nums[left]] == 0)
                    k++;
                left++;
            }


            count += (right - left + 1);
        }
        return count;
```

```
    }

    int subarraysWithKDistinct(vector<int>& nums, int k) {
        return atMostK(nums, k) - atMostK(nums, k - 1);
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1,2,1,2,3};
    int k = 2;
    cout << sol.subarraysWithKDistinct(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each element is processed at most twice using sliding window.
- **Space Complexity:** `O(K)`
  The frequency map stores up to k distinct integers.

11. Smallest Window in a String Containing All Characters of Another String

Problem recap

You are given two strings s and p.

You need to find the **smallest substring of s** that contains **all characters of p including duplicates**.

Rules:

- If no such substring exists, return `""`
- If multiple substrings have the same minimum length, return the one with the **smallest starting index**

Example 1

Input

`s = "timetopractice", p = "toc"`

Output

`"toprac"`

Explanation

The substring `"toprac"` is the smallest window where characters `t`, `o`, and `c` all appear.

Example 2

Input

`s = "zoomlazapzo", p = "oza"`

Output

`"apzo"`

Naive Approach

Algorithm

1. Generate **all possible substrings** of s.
2. For each substring:
    - Check if it contains **all characters of p with correct frequencies**.
3. Track the substring with the **minimum length** that satisfies the condition.
4. Return the smallest such substring.

Code

```
#include <climits>

#include <iostream>

#include <string>

using namespace std;


bool hasAllChars(string &sub, string &p) {

    int count[256] = {0};


    for (char ch : p)

        count[ch]++;


    for (char ch : sub) {
```

```
        if (count[ch] > 0)
            count[ch]--;
    }

    for (int i = 0; i < 256; i++) {
        if (count[i] > 0)
            return false;
    }
    return true;
}

string minWindow(string &s, string &p) {
    int n = s.length();
    int minLen = INT_MAX;
    string res = "";

    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            string sub = s.substr(i, j - i + 1);
            if (hasAllChars(sub, p)) {
                if (sub.length() < minLen) {
                    minLen = sub.length();
                    res = sub;
```

```
                }
            }
        }
    }
    return res;
}

int main() {
    string s = "timetopractice";
    string p = "toc";
    cout << minWindow(s, p);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n^3)$
- **Space Complexity:** $O(1)$ (fixed size frequency array)

Better Approach (Binary Search on Answer)

Algorithm

1. The minimum window length can range from `|p|` to `|s|`.
2. Use **binary search** on window size.
3. For a given window size `mid`:
    - Check if **any substring of size `mid`** contains all characters of p.
4. If valid, try smaller window sizes.
5. Otherwise, try larger sizes.
6. Store the smallest valid window found.

Code

```
#include <iostream>

#include <string>

#include <climits>

using namespace std;


bool isValid(string &s, string &p, int mid, int &start) {

    int count[256] = {0};

    int distinct = 0;


    for (char c : p) {

        if (count[c] == 0) distinct++;

        count[c]++;

    }
```

```
    int matched = 0;
    for (int i = 0; i < s.size(); i++) {
        count[s[i]]--;
        if (count[s[i]] == 0)
            matched++;

        if (i >= mid) {
            count[s[i - mid]]++;
            if (count[s[i - mid]] == 1)
                matched--;
        }

        if (i >= mid - 1 && matched == distinct) {
            start = i - mid + 1;
            return true;
        }
    }
    return false;
}

string smallestWindow(string s, string p) {
    int low = p.length(), high = s.length();
```

```
    int minLen = INT_MAX, idx = -1;

    while (low <= high) {
        int mid = (low + high) / 2;
        int start;
        if (isValid(s, p, mid, start)) {
            minLen = mid;
            idx = start;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    if (idx == -1) return "";
    return s.substr(idx, minLen);
}

int main() {
    string s = "timetopractice";
    string p = "toc";
    cout << smallestWindow(s, p);
    return 0;
```

```
}
```

Complexity Analysis

- **Time Complexity:** `O(n log n)`
- **Space Complexity:** `O(1)`

Expected Approach (Sliding Window)

Algorithm

1. Use **two pointers** (`start`, `end`) to maintain a sliding window on `s`.
2. Maintain two frequency arrays:
    - One for characters in p
    - One for characters in the current window
3. Expand the window by moving `end`.
4. Once all characters of p are present:
    - Shrink the window from the left to minimize it.
5. Keep updating the smallest valid window found.
6. Return the smallest window.

Code

```
#include <iostream>
```

```cpp
#include <string>
#include <vector>
#include <climits>
using namespace std;

string smallestWindow(string s, string p) {
    int len1 = s.length(), len2 = p.length();
    if (len1 < len2) return "";

    vector<int> countP(256, 0), countS(256, 0);

    for (char c : p)
        countP[c]++;

    int start = 0, start_idx = -1, min_len = INT_MAX;
    int matched = 0;

    for (int end = 0; end < len1; end++) {
        countS[s[end]]++;

        if (countP[s[end]] != 0 && countS[s[end]] <= countP[s[end]])
            matched++;
```

```
        if (matched == len2) {
            while (countS[s[start]] > countP[s[start]] ||
countP[s[start]] == 0) {
                if (countS[s[start]] > countP[s[start]])
                    countS[s[start]]--;
                start++;
            }

            int window_len = end - start + 1;
            if (window_len < min_len) {
                min_len = window_len;
                start_idx = start;
            }
        }
    }

    if (start_idx == -1) return "";
    return s.substr(start_idx, min_len);
}

int main() {
    string s = "timetopractice";
    string p = "toc";
```

```
    cout << smallestWindow(s, p);

    return 0;

}
```

Complexity Analysis

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$ (fixed size arrays)

12. Minimum Window Subsequence

Problem recap
Given strings s1 and s2, find the **smallest contiguous substring of s1** such that s2 appears **as a subsequence** inside it (order preserved, not necessarily consecutive).
If multiple windows have the same minimum length, return the **leftmost** one. If none exists, return `""`.

---

1) Naive Approach — Brute Force

Idea: Try all substrings of s1 and check if s2 is a subsequence of each.

Algorithm

1. Generate every substring `s1[i..j]`.

2. Check if s2 is a subsequence of that substring.
3. Track the smallest valid substring.

Complexity

- **Time:** $O(n^3)$ (substrings × subsequence check)
- **Space:** $O(1)$

Code

```
#include <iostream>
#include <string>
#include <climits>
using namespace std;

bool isSubsequence(string &sub, string &s2) {
    int i = 0, j = 0;
    while (i < sub.size() && j < s2.size()) {
        if (sub[i] == s2[j]) j++;
        i++;
    }
    return j == s2.size();
}

string minWindow(string &s1, string &s2) {
```

```
    int n = s1.size();
    string ans = "";
    int minLen = INT_MAX;

    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            string sub = s1.substr(i, j - i + 1);
            if (isSubsequence(sub, s2)) {
                if ((int)sub.size() < minLen) {
                    minLen = sub.size();
                    ans = sub;
                }
                break; // smallest for this start
            }
        }
    }
    return ans;
}

int main() {
    string s1 = "abcdebdde", s2 = "bde";
    cout << minWindow(s1, s2) << endl; // bcde
}
```

---

2) Better Approach — Two Pointers + Backtracking

Idea:

- From each index where `s1[i] == s2[0]`, scan forward to match s2.
- Once matched, **backtrack** to shrink the window as much as possible.

Algorithm

1. For each `i` where `s1[i] == s2[0]`:
    - Move forward pointers to match all of s2.
    - Backtrack from the end to minimize the window.
2. Keep the smallest window.

Complexity

- **Time:** `O(n²)`
- **Space:** `O(1)`

Code

```
#include <iostream>

#include <climits>

#include <string>

using namespace std;
```

```
string minWindow(string &s1, string &s2) {
    int n = s1.size(), m = s2.size();
    string ans = "";
    int minLen = INT_MAX;

    for (int i = 0; i < n; i++) {
        if (s1[i] != s2[0]) continue;

        int p1 = i, p2 = 0;
        while (p1 < n && p2 < m) {
            if (s1[p1] == s2[p2]) p2++;
            p1++;
        }

        if (p2 == m) {
            int end = p1 - 1;
            p2 = m - 1;
            while (end >= i) {
                if (s1[end] == s2[p2]) p2--;
                if (p2 < 0) break;
                end--;
            }
            int start = end;
```

```
            int len = p1 - start;
            if (len < minLen) {
                minLen = len;
                ans = s1.substr(start, len);
            }
        }
    }
    return ans;
}

int main() {
    string s1 = "abcdebdde", s2 = "bde";
    cout << minWindow(s1, s2) << endl; // bcde
}
```

---

3) Expected / Optimal Approach — Next Occurrence Preprocessing

Idea:
Precompute where the **next occurrence** of each character appears in s1.
Then, for any start index, we can jump quickly to match s2.

Algorithm

1. Build `nextPos[i][c]`: index of next occurrence of char c at or after `i`.

2. For each valid start, jump through `nextPos` to match all chars of `s2`.
3. Track the minimum window.

Complexity

- **Time:** `O(n × m)`
- **Space:** `O(n)` (next occurrence table)

Har possible start se try karo ki `s2` ko subsequence ke form me complete kar sakte ho ya nahi, aur fast jumping ke liye `nextPos` table use karo.

Code

```
#include <iostream>
#include <vector>
#include <string>
#include <climits>
using namespace std;

string minWindow(string &s1, string &s2) {
    int n = s1.size(), m = s2.size();
    vector<vector<int>> nextPos(n + 1, vector<int>(26, -1));

    for (int c = 0; c < 26; c++)
        nextPos[n][c] = -1;
```

```
for (int i = n - 1; i >= 0; i--) {
    nextPos[i] = nextPos[i + 1];
    nextPos[i][s1[i] - 'a'] = i;
}

string ans = "";
int minLen = INT_MAX;

for (int start = 0; start < n; start++) {
    if (s1[start] != s2[0]) continue;

    int idx = start;
    bool ok = true;

    for (int j = 0; j < m; j++) {
        if (idx == -1) { ok = false; break; }
        idx = nextPos[idx][s2[j] - 'a'];
        if (idx == -1) { ok = false; break; }
        idx++;
    }

    if (ok) {
        int endIdx = idx - 1;
```

```
            int len = endIdx - start + 1;
            if (len < minLen) {
                minLen = len;
                ans = s1.substr(start, len);
            }
        }
    }
    return ans;
}

int main() {
    string s1 = "abcdebdde", s2 = "bde";
    cout << minWindow(s1, s2) << endl; // bcde
}
```

—

```
for (int i = n - 1; i >= 0; i--) {
   nextPos[i] = nextPos[i + 1];
   nextPos[i][s1[i] - 'a'] = i;
}
```

Isko todte hain

Step 1: Peeche wali info copy

nextPos[i] = nextPos[i + 1];

Matlab:

“Agar maine is position se kuch nahi badla,

to jo aage se milta tha, wahi yahan se bhi milega.”

Step 2: Current character update

nextPos[i][s1[i] - 'a'] = i;

Matlab:

“Is index par ye character khud mil raha hai,

to iska next occurrence yahi hai.”

nextPos use kyun ho raha hai?

Code me ye line dekho:

idx = nextPos[idx][s2[j] - 'a'];

Iska matlab:

“Current position se,

s2[j] next kahan milega?”

Mil gaya → jump

Nahi mila → fail